

1. プロジェクト概要

本プロジェクトは、リアルな占い体験を提供するワンオラクル(1枚引き)タロット占いアプリの開発である。対面の占い師に占ってもらっているような没入感のあるアニメーション演出を特徴とする。本フェーズではMVP(Minimum Viable Product)として、ゲストユーザー向けの大アルカナのみを使用した1枚引き機能を実装する。将来的なユーザー認証、履歴保存、AI解説、課金機能の追加を見据えた拡張性の高い設計とする。

2. システムアーキテクチャ(現状維持)

- バックエンド: Cloudflare Workers + Hono
- データベース: Prisma (PostgreSQL または Supabase)
- フロントエンド: React 19 + TypeScript + Vite
- アセット管理: WebP画像 + Lottieフォールバック

3. スコープ定義

今回のフェーズ(MVP)に【含める】もの

- 大アルカナ22枚のデータ完備(キーワードの追加)
- トップ画面のリッチUI実装
- 対面占い風のシャッフル〜カット〜ドローの一連のアニメーションフロー
- ランダムロジックによるワンオラクル(1枚引き)機能
- 結果表示画面(カード画像+キーワード)
- ゲスト利用を前提としたフロー

今回のフェーズから【除外】するもの(将来フェーズ)

- 小アルカナ56枚の実装
- 複数枚引き(スプレッド)対応
- ユーザー認証(ログイン機能)
- 占い履歴のDB保存
- AI API連携による詳細リーディング解説機能
- 広告掲載・課金システム

4. データベース設計要件(拡張性の確保)

将来の履歴保存やスプレッド対応を見据え、現在の **TarotCard** モデルを維持しつつ、後続フェーズで **ReadingHistory** や **User** テーブルを容易にリレーションできる構造にしておく。

【タロットカードデータ要件】

- 対象: 大アルカナ22枚

- カラム追加/更新: 正位置キーワード(配列またはカンマ区切り文字列で2〜3個)、逆位置キーワード(同左)を格納できるようにする。
- ※既存の「未設定」プレースホルダーを実データに置き換える。

5. 機能およびUI/UX要件

占いフローは以下の順序で遷移する。内部ロジックは「完全ランダム」とし、演出(アニメーション)によってユーザーの納得感と体験価値を高める。

① トップ画面

- **UI:** 「スタート」ボタンのみを中央に配置。
- 背景演出: 「寿司打」のような、リッチ感がありつつ動きのある動的背景(パーティクルアニメーション、またはCSS/Canvasによるリッチな描画)。

② シャッフル画面(占い師対面視点)

- トリガー: トップ画面の「スタート」押下。
- 映像/アニメーション: 画面の向こう側に占い師が座っている視点。テーブル上でカードを両手でぐるぐるかき混ぜる(ウォッシュシャッフル)アニメーションをループ再生する。
- **UI:** 画面中央に「占ってもらふ事柄を決めてください。」というテキストメッセージを表示。画面下部に「決まりました」ボタンを配置。

③ カット(山分け)画面

- トリガー: 「決まりました」ボタン押下。
- アニメーション: かき混ぜていたカードを1つの山にまとめる(※トントンと揃える動作があればベター。優先度は低)。
- カット手順1(3分割):
 - カードの山を3つに分けるアニメーション。
 - ユーザーに「重ねる順番」を選択させるUI(タップ順など)。
 - 選択された順に山が重なるアニメーション。
- カット手順2(2分割):
 - 再度、山を2つに分ける。
 - ユーザーに「重ねる順番」を選択させる。
 - 選択された順に山が重なる。
- 向きの決定:
 - まとまった山を横向きに提示する。
 - ユーザーに「どちらを上にしてカードを開くか(右か左か)」を選択させるUI。

④ 結果表示画面

- ドローロジック: カットや向きの決定等のユーザーアクションにかかわらず、バックエンドまたはフロントエンドのロジックにて完全ランダムで1枚(カード種類と正逆)を決定する。
- 演出: 選ばれたカードがめくられるアニメーション。
- **UI:**
 - カードの画像を大きく表示(WebP)。

- 選ばれた状態(正位置 or 逆位置)に応じた「キーワード(2~3つ)」を表示。
- トップへ戻るボタン。

6. 将来拡張への留意点(テクニカルノート)

AIへの指示として、以下の点を考慮してコードを設計すること。

- **API設計:** 今回は1枚引きだが、将来的に `GET /api/draw?count=3` のように引く枚数を指定できるよう、拡張性のあるエンドポイント設計にしておく。
- **コンポーネント設計:** シャッフルやカットのアニメーション部分はコンポーネントとして独立させ、将来別のスプレッドを導入する際にも再利用しやすくする。
- **状態管理:** 現在はゲスト状態(ローカルステートのみ)で進行するが、将来的にContext APIやRedux、またはZustand等にユーザーセッションや履歴データを持たせやすいように疎結合な設計を心がける。